

MobiShopy - Mobile Shop Management System

Course: Advanced Database Management System Practical

Database: MySQL

Project Description

The Mobile Shop Management System (MSMS) is being built as a database project for a single mobile shop, covering the shop's customers, suppliers, mobile inventory and day-to-day sales and billing. The frontend is developed using HTML, CSS and JavaScript, and the same self-created dataset is used to work through the database design, population and querying stages covered in the ADBMS practical.

Objectives

1. To enable the design of a structured dataset using proper data modelling techniques.
2. To develop practical skills in SQL for database creation, manipulation and querying.
3. To enhance analytical thinking through multi-phase database development.

Phase 1 - Data Modelling and Dataset Design

1. **Dataset:** A small mobile shop dataset covering customers, suppliers, mobiles and sales records. The same data will be carried forward and used in all later phases.
2. **Entity and attribute identification:** CUSTOMER, SUPPLIER, MOBILE and SALES are treated as the main entities, each with the attributes needed to store their respective details.
3. **ER modelling:** SUPPLIER-MOBILE, CUSTOMER-SALES and MOBILE-SALES will be modelled mainly as 1:M relationships.
4. **Relational schema:** Convert the ER model into tables. Primary keys will identify records and foreign keys will connect the related tables.
5. **Normalization:** customer, supplier, mobile and sales details will be kept in separate tables so that repeated data is reduced and the schema stays in 3NF.

ER Diagram

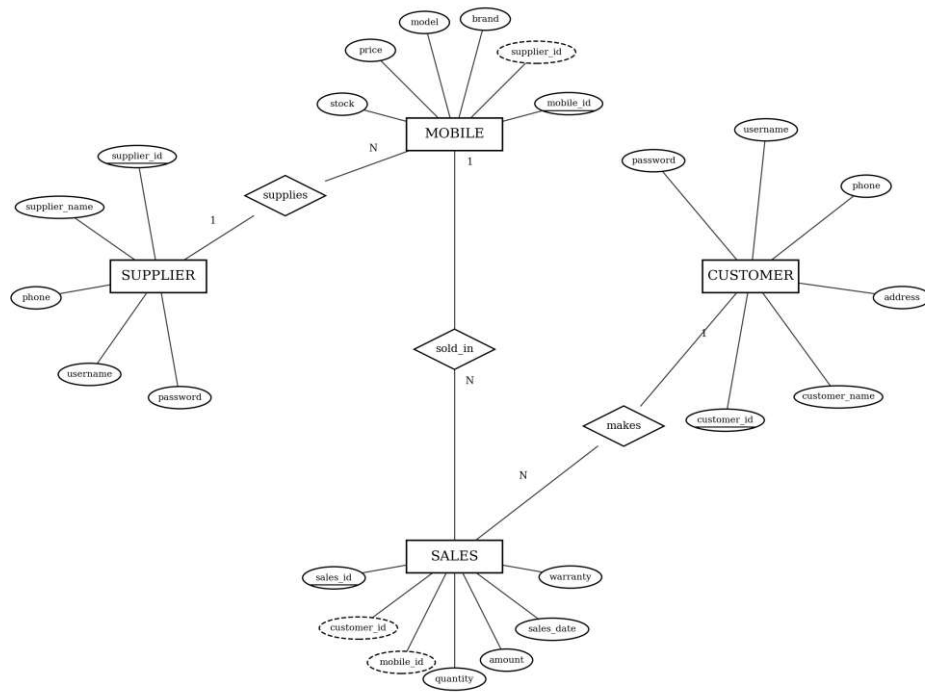


Figure: Chen-notation ER diagram for the Mobile Shop Management System

Tables and Data Types

Table	Fields with Data Type	Relationship
CUSTOMER	customer_id INT PK AUTO_INCREMENT customer_name VARCHAR(100) NOT NULL address VARCHAR(255) NOT NULL phone VARCHAR(15) NOT NULL UNIQUE username VARCHAR(50) NOT NULL UNIQUE password VARCHAR(255) NOT NULL	CUSTOMER 1:M SALES
SUPPLIER	supplier_id INT PK AUTO_INCREMENT supplier_name VARCHAR(100) NOT NULL phone VARCHAR(15) NOT NULL UNIQUE username VARCHAR(50) NOT NULL UNIQUE password VARCHAR(255) NOT NULL	SUPPLIER 1:M MOBILE
MOBILE	mobile_id INT PK AUTO_INCREMENT supplier_id INT FK brand VARCHAR(50) NOT NULL model VARCHAR(100) NOT NULL price DECIMAL(10,2) NOT NULL stock INT NOT NULL DEFAULT 0	FK supplier_id -> SUPPLIER MOBILE 1:M SALES
SALES	sales_id INT PK AUTO_INCREMENT customer_id INT FK mobile_id INT FK	FK customer_id -> CUSTOMER FK mobile_id -> MOBILE

Table	Fields with Data Type	Relationship
	quantity INT NOT NULL amount DECIMAL(10,2) NOT NULL sales_date DATETIME NOT NULL warranty VARCHAR(50)	

Relational Schema: SUPPLIER(supplier_id) -> MOBILE(supplier_id); CUSTOMER(customer_id) -> SALES(customer_id); MOBILE(mobile_id) -> SALES(mobile_id).

Phase 2 - Database Creation and Data Preparation

1. **DDL:** Create mobile_shop_db and all four tables using CREATE DATABASE and CREATE TABLE. PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL and CHECK constraints will be defined along with the tables.
2. **DML:** Sample data will be entered using INSERT. UPDATE will be used whenever price or stock changes, while DELETE will only be tested where foreign-key relations are not affected.
3. **Data cleaning and constraints:** duplicate phone numbers and usernames will be prevented, required IDs will not be left NULL, and invalid negative price or stock values will be restricted.
4. **NULL handling:** warranty can stay NULL when a mobile is sold without a warranty plan. IS NULL / IS NOT NULL will be used to locate such records.
5. **Basic queries:** SELECT, WHERE and ORDER BY will be used to display mobiles currently in stock, recent sales, mobiles by brand and mobiles sorted by price.
6. **Aggregate functions:** COUNT() will calculate the number of sales per mobile, AVG() will calculate the average sale amount, and SUM() will calculate total revenue per supplier.

Phase 3 - Data Querying and Analytical Operations

1. **Joins:** join CUSTOMER + SALES + MOBILE to generate a bill and display purchase history, and SUPPLIER + MOBILE to view the mobiles supplied by each supplier.
2. **Nested queries and subqueries:** find mobiles priced above the average mobile price, and find customers whose total purchase amount is above the overall average.
3. **View:** create Mobile_Sales_Summary to display mobile name, brand, total quantity sold and total revenue through one reusable query.
4. **Query execution and optimization:** use EXPLAIN on the customer-sales-mobile join. Indexes will be created on frequently searched columns such as SALES.customer_id, SALES.mobile_id and MOBILE.supplier_id, and the query plan will be checked again.
5. **Function:** create GetMobileSalesCount(mobile_id). It will receive a mobile ID, count the matching records in SALES and return the total units sold for that mobile.
6. **Procedure:** create AddSale(customer_id, mobile_id, quantity). It will accept the sale details, check the available stock, insert a new SALES record and reduce the mobile's stock.

Functional Modules

Function / Task	Tables Used	How I will do it
Register customer / supplier	CUSTOMER, SUPPLIER	I will store customer and supplier details separately, each with a unique username for login.
Add and manage mobiles	MOBILE, SUPPLIER	I will link every mobile to the supplier_id that supplies it, and update the price or stock as required.
Search, filter and sort mobiles	MOBILE	I will use WHERE for searching and filtering by brand, price and stock, along with ORDER BY and an application-level Merge Sort for sorting.
Purchase a mobile	CUSTOMER, MOBILE, SALES	I will check the available stock, insert a SALES record and reduce the mobile's stock, all within one transaction.
Generate bill	CUSTOMER, MOBILE, SALES	I will join these three tables to prepare a bill dynamically, without maintaining a separate bill table.
View purchase history	CUSTOMER, SALES, MOBILE	I will join these tables so that a customer can view only their own past purchases.
View supplied mobiles	SUPPLIER, MOBILE	I will filter the mobile table by supplier_id so a supplier can see their own listings and stock.

Phase 4 - Data Integrity and Transaction Handling

1. **Transaction handling:** when a customer buys a mobile, I will start one transaction, insert the SALES record and reduce MOBILE.stock. A SAVEPOINT will be placed before the stock update. COMMIT will run if both operations succeed, and ROLLBACK if an error occurs.
2. **Atomicity:** the sales insertion and stock update will be treated as one unit. If either operation fails, the complete transaction will be rolled back so the database is never left partly updated.
3. **Concurrency case:** before confirming a purchase, I will check that the requested quantity is still available inside the transaction, which helps prevent the same stock from being sold twice.
4. **Integrity constraints:** foreign keys will ensure that MOBILE always references an existing SUPPLIER, and that SALES always references an existing CUSTOMER and MOBILE.
5. **Final validation:** check for missing references, confirm that mobile stock matches the recorded sales, and re-run the basic queries, joins, subqueries, view, function, procedure and transaction cases to verify the database.

Outcome of the Project

1. A normalized relational database for mobile shop records, up to 3NF.
2. Practical implementation of DDL, DML, constraints, basic queries, aggregates, joins, subqueries, views, functions/procedures and transactions.
3. Customer, supplier, mobile and sales information can be stored and retrieved through related tables.
4. The project gives hands-on understanding of data modelling, SQL querying, integrity and transaction handling, built around an HTML, CSS and JavaScript frontend for the shop.